Chapter 1

Background and Related Work

Multirotors flying in close proximity disturb one another aerodynamically. The disturbance is large enough to destabilise a vehicle, and conventional practice avoids it by keeping the vehicles far apart: a Crazyflie 2.0 measures 9 cm rotor-to-rotor, and when the interaction between vehicles is left unmodelled, planners have adopted conservative separations, Shi et al. [15] cite 60 cm from earlier planning work as one such example. Removing that margin is what makes dense formation flight possible, and doing so requires a controller that accounts for the interaction rather than avoiding it.

This chapter surveys how that has been attempted. It is organised as an argument rather than a catalogue. Section 1.2 establishes what the disturbance is; 1.3 and 1.4 describe the two sources of information a controller can have about it, namely online measurement and offline prediction; 1.5 shows that a prediction is only useful once it enters a control law, and that how it enters differs by controller family; 1.6 covers the alternative of learning the corrective action directly; 1.7 covers combining measurement and prediction. Section 1.8 then examines how these methods are evaluated, and argues that the field's reporting conventions make the individual results impossible to compose into a comparison. That observation is the gap this thesis addresses.

The survey covers the field as a whole, including families this thesis does not itself implement. Where a family falls outside the strategies evaluated here — learning the corrective action directly (Section 1.6) and embedding the residual in a predictive-control horizon (Section 1.5) — it is described briefly rather than in depth. The omission is one of scope, not of relevance: those families are part of the landscape the gap argument of Section 1.8 rests on, and Chapter **??** states where the boundary of this thesis's own comparison falls and why.

1.1 Multirotor dynamics and baseline control

Each vehicle i in a team is modelled as a rigid body,

$$\dot{p}_i = v_i, \qquad m\,\dot{v}_i = -mge_3 + f_i R_i e_3 + f_{\mathrm{res},i}, \tag{1.1}$$

$$\dot{R}_i = R_i \hat{\omega}_i, \qquad J\dot{\omega}_i = -\omega_i \times J\omega_i + \tau_i + \tau_{\mathrm{res},i}, \tag{1.2}$$

in a world frame with e_3 pointing up, where f_i and τ_i are the collective thrust and body torque produced by the rotors. The terms $f_{\mathrm{res},i}$ and $\tau_{\mathrm{res},i}$ collect every force and moment the nominal model omits. This chapter is concerned with the case in which they are dominated by the aerodynamic interaction with neighbouring vehicles. The formulation is developed in Chapter **??**.

The baseline against which interaction-aware methods are measured is a geometric tracking controller on $SE(3)$, which tracks a position trajectory and a single heading direction coordinate-free, avoiding the representation singularities of a local attitude parameterisation [11]. Its guarantees are regional rather than global: exponential stability for initial attitude errors below 90 °, and almost-global exponential attractiveness below 180 °. It is the controller three of the strategies compared in this thesis build upon, and the one used uncompensated as a reference condition. The attitude error function is that work's, verified equation-by-equation against its companion arXiv paper [10]; the torque law implemented here is a *reduced* form of it, and Chapter **??** states exactly which terms differ.

A second property of the quadrotor is used throughout: differential flatness. Position and yaw, together with their derivatives, determine the full state and the inputs, which allows a feedforward term to be computed directly from a planned trajectory. Tal and Karaman [18] exploit this to track position and yaw derivatives up to fourth order, using flatness-derived feedforward for angular rate and angular acceleration in order to follow jerk and snap. Flatness recurs in Section 1.5 as a structure that a learned correction can either preserve or destroy.

1.2 Aerodynamic interaction in multirotor teams

The dominant interaction in a vertically stacked formation is the downwash of the upper vehicle acting on the lower one. Two properties make it difficult to handle. It is strongly nonlinear, growing sharply as the vehicles close and decaying quickly with lateral offset; and it is asymmetric, since the vehicle above is affected far less than the one below.

Recent experimental work characterises the effect directly rather than inferring it from tracking error. Kiran et al. [9] measure forces *and* torques with load cells on mounted Crazyflies whose roll and yaw are locked and whose motors are driven at a constant hover voltage, and use particle image velocimetry in a centre-plane light sheet to resolve the wake, for a single quadrotor and for a pair; the dynamic cases traverse one vehicle on a rail rather than flying it. Gielis et al. [5] measure six-degree-of-freedom exogenous forces on a multirotor held on a purpose-built load stand at hover thrust, while up to three further vehicles fly freely around it, and report strong nonlinearities in how those interactions aggregate. The vehicle experiencing the disturbance is therefore constrained rather than in free flight. Both measure the disturbance as a transducer load rather than inferring it from a flight controller's residual, actuation is still present in each, at hover thrust, which makes them, *in our reading*, the natural ground truth against which a learned model should be checked. Neither paper makes that claim, and neither validates a learned model of ours or anyone else's.

Simplified physical models exist. Bauersfeld et al. [2] analyse roughly 16 h of hover data across six platforms spanning 0.23–6.3 kg, and show that once the individual propeller wakes have merged the mean flow is well approximated by a turbulent jet — beyond about 2.5 motor-to-motor distances l below the vehicle, a length scale the abstract renders as "drone-diameters". That gives a computationally light alternative to computational fluid dynamics. What it estimates is the *time-averaged magnitude* of the induced flow *in hover*, not an instantaneous six-degree-of-freedom wrench on a follower. For the near field the authors report no closed-form first-principles model and point to computational fluid dynamics instead. Formation flight operates in exactly that near field — our observation, not theirs — which is the practical argument for learning the residual from data rather than deriving it.

1.3 Reactive compensation: measuring the residual online

The first family estimates the residual from its effect and requires no prior model of the interaction. Incremental nonlinear dynamic inversion (INDI) compares measured acceleration with the acceleration the nominal model predicts for the actuation actually applied, and attributes the difference to unmodelled forces, which it then cancels incrementally.

Smeur et al. [16] cascade an inner angular-acceleration INDI loop with an outer linear-acceleration INDI loop on a single Parrot Bebop running Paparazzi at 512 Hz; the position and velocity loops feeding the acceleration reference remain proportional-derivative rather than a third incremental stage. They demonstrate disturbance rejection indoors by flying in and out of a 10 m/s open-jet exhaust every 14 s. In two *separate* experiments they show the method does not depend on frequent position updates: indoors with motion capture deliberately downsampled to 4 Hz, and outdoors taking off and holding position on a consumer GNSS module in roughly 5.1 m/s wind. Tal and Karaman [18] use INDI for robust tracking of linear and angular accelerations in aggressive flight on a single custom 609 g vehicle, and state the property that defines the family: no parametric model of the aerodynamic effects is required. That is narrower than it sounds, mass, inertia, motor time constant, control effectiveness and the throttle map are all still identified; it is the *drag* model that is dispensed with.

The same work also exposes the family's cost. Tracking snap requires direct control of body torque, which those authors achieve using closed-loop motor speed control based on optical encoders on their own vehicle's motors [18]; encoders are what *that* snap-and-torque path required, not a general prerequisite for INDI. Reconstructing the applied wrench well enough to difference it against measurement does, however, generally require some form of rotor-speed feedback, Smeur et al. likewise use motor-speed measurement [16], which is additional hardware or firmware support that must be fitted, calibrated and maintained on every vehicle. A second limitation is structural rather than practical: the estimate is delayed by the sensing and filtering chain, and it exists only once the disturbance has already perturbed the vehicle. A reactive controller cannot, even in principle, act before the disturbance arrives.

That the sensing requirement is not fundamental has since been shown. Cobo-Briesewitz et al. [4] demonstrate that a neural network can generate smooth approximations of INDI's residual estimate without rotor-speed measurement, which is discussed in Section 1.7.

1.4 Predictive compensation: learning the residual from relative state

The second family predicts the residual before it acts, from the relative states of the neighbours, using a model learned offline from flight data. Because the interaction is a function of the team's relative configuration, a model with the right inputs can act ahead of the disturbance rather than after it.

Shi et al. [15] combine a nominal dynamics model with a regularised permutation-invariant deep neural network that learns multi-vehicle interaction, and use the learned model as a feed-forward term in a decentralised nonlinear tracking position controller; geometric $SE(3)$ control appears only as an optional attitude inner loop, and the learned force itself is the vertical component $f_{a,z}$, evaluated onboard the STM32. Training separate networks on data from two, three and four vehicles respectively, they report worst-case height error reduced by a factor of two or better, and up to four times smaller in the most favourable case. In a two-vehicle swapping task the baseline reaches 9.4 cm; the network

trained on *four* vehicles brings this to 2.4 cm, while the network trained on two vehicles reaches 2.7 cm. The swap experiments use vertical neighbour spacings of 0.20–0.25 m, with 25 cm stated explicitly for the three-vehicle case. Generalisation is directional rather than universal: networks trained on three or four vehicles improved five-vehicle flight, whereas a network trained only on two vehicles did *not* generalise, performing worse than the baseline on three- and four-vehicle tests.

Neural-Swarm2 [14] extends this to heterogeneous teams using heterogeneous deep sets, whose permutation-invariant architecture accepts any number of neighbours. That is a structural property rather than an accuracy claim: the networks are trained on at most three vehicles (at most two neighbours), and the paper describes prediction at sixteen robots as relatively less accurate. Two design choices are worth noting. Spectral normalisation is applied, which the abstract describes as yielding stability and generalisation guarantees; in the body this rests on Lipschitz bounds together with empirical generalisation and a bounded-residual assumption used in the proofs, rather than a formal guarantee on unseen data. The learned residual is used in interaction-aware motion planning as well as in tracking control. They report flying a heterogeneous team of sixteen robots at a minimum vertical distance of 24 cm, and, separately, up to a threefold reduction in worst-case tracking error against a baseline nonlinear tracking controller with the interaction force set to zero. The two figures are reported in the abstract and we do not assume they describe the same experiment: on the sixteen-robot flight itself the body reports the maximum z-error of a small vehicle falling from 15 cm to 10 cm, a factor of about 1.5.

Subsequent work has focused on making these models cheaper to obtain rather than larger. Smith et al. [17] exploit the rotational symmetry of the problem with an $SO(2)$-equivariant model, and report that five minutes of real-world flight data attains a lower validation error than learning baselines trained on more than fifteen — a data-efficiency result on prediction accuracy rather than a closed-loop tracking one. Deployed online within an optimal feedback (LQR) controller on two identical vehicles, the model reduces three-dimensional tracking error by 36 % and vertical tracking error by 56 % on average.[1] On a lateral transect flown beneath the hovering vehicle they report mean three-dimensional position error falling from 0.154 m to 0.098 m, close to 36 %, with vertical improvements of 55 % and 49 %. The platform is not a Crazyflie but a custom quadrotor of roughly 0.26 m span and 0.7 kg, tracked by OptiTrack and controlled from a Raspberry Pi at 50 Hz with a 45 Hz network update. The closest *training* stage is about 0.5 m — some two body-lengths — while the evaluation manoeuvres are flown at 0.6–0.8 m, together with a lemniscate flown beneath the hovering vehicle at $z = -2.5\,m$. The authors attribute the efficiency to exploiting the problem's latent geometry rather than learning it from data.

The superposition assumption The deep-sets architectures above aggregate per-neighbour terms by summation, which commits them to a form of superposition. The literature is explicit that this is an approximation. Shi et al. [15] observe that with more than two vehicles the aerodynamic effect is not a simple superposition of each pair. Gielis et al. [5] make the same point as their motivation, an accurate model of the downwash from one vehicle is unlikely to suffice for predicting the aggregate from several. Their abstract frames this as modelling the formation as a graph; the implementation learns the aggregation function with a sum-pooling deep-sets predictor, contrasted against a

[1] The paper is internally inconsistent on whether the 36 % figure is three-dimensional or lateral: we follow the abstract and the translation-result body, which give three-dimensional and vertical. Its contributions list instead calls the 36 % figure lateral.

linear-summation baseline. Generalisation to a fourth vehicle is claimed at abstract level, while the reported tables stop at three neighbours and the conclusion is pessimistic about extrapolating further, so we do not treat it as a demonstrated held-out result.

Their measurements refine the picture rather than simply refuting summation. They report that interactions fall into three regimes depending on the formation and its location in the state space: a chaotic, noise-dominated regime; a linear regime *well estimated by linear summation*; and a nonlinear regime requiring a nonlinear model. Superposition is therefore not uniformly wrong but regime-dependent, which makes the practical question one of how much accuracy is lost, and where. This is their own load-stand result, obtained independently of Shi et al.'s free-flight observation cited above.

Summed architectures nevertheless remain in wide use, because a controller does not require an exact model, only one whose remaining residual is smaller than the one it removes. What is not reported is the size of the penalty that the simplification carries for a deployed model, which is the question this thesis takes up in the multi-robot results chapter.

1.5 Injecting the prediction: the controller matters

A predicted residual is inert until it enters a control law, and the manner of entry differs by controller family. Treating "predictive compensation" as a single method would obscure differences that are as large as those between the families themselves.

Feedforward into a geometric controller. The simplest injection adds the predicted force to the desired force of the baseline controller, leaving its structure unchanged. This is the form used with the permutation-invariant models of Section 1.4, whose baseline is a nonlinear tracking controller [15, 14].

Feedback linearisation with a flatness-preserving residual. A learned correction added to a nominal model can destroy differential flatness, and with it the applicability of flatness-based planning and control. Yang et al. [19] show that residuals with a lower-triangular structure preserve both the local flatness of the system and its original flat output for systems whose nominal model admits a pure-feedback form, and give a constructive procedure for recovering the diffeomorphism of the augmented system. The residual there is a *structured correction inside the state derivative* rather than a free six-degree-of-freedom wrench — in their worked example it appears only in the velocity equation. Their validation is in simulation on a planar, two-dimensional quadrotor; there is no hardware, no three-dimensional vehicle and no multi-robot downwash experiment, and they list a physical quadrotor as future work. Hsieh et al. [8] apply the idea to tight formation flight, learning a physics-informed residual that keeps the joint multi-quadrotor system differentially flat and using the preserved structure to design a feedback-linearising controller that cancels the disturbance by feedforward. Their abstract reports a 31 % reduction in average tracking error against nominal baselines; the body does not restate that aggregate, reporting instead, on hardware, 24 % RMSE and 30 % maximum-z improvement for ROM-FBL over nominal FBL, and a further 29 % RMSE and 43 % maximum-z improvement for learned FBL over ROM-FBL. On hardware they match the approximately 5 cm RMSE and 10 cm maximum-z error *published* by Chee et al. [3], rather than comparing the two methods on one rig; in their simulation study the flatness-based controller's maximum-z error can be worse than that of nonlinear model predictive control. The 20-fold reduction

in solve time (about 1 ms against 27 ms at equal residual) is a *simulation* figure; the introduction separately quotes a sevenfold compute reduction, and the abstract an order of magnitude. Training uses 28 s of hardware data and 18 s in simulation. Two qualifications matter for our purposes: the hardware experiments use two Crazyflie 2.1 quadrotors with thrust-upgrade bundles — very nearly the platform used in this thesis — and the controller runs *off-board*, on an Intel i5 laptop streaming commands over a Crazyradio link at 400 Hz, rather than on the vehicle. Their 5 ms loop is therefore an abstract-level, off-board figure, and the claim it supports is that the method is cheap enough to be worth porting, not that it has been shown to run onboard.

On hardware they also compensate residual *forces* only, commanding thrust and angular velocity, and delegating residual torques to the vehicle's own rate controller, the same scoping decision taken in Chapter **??**. Their simulation study does still model residual torques.

Inside a model predictive control horizon. The residual can instead form part of the prediction model that an optimiser reasons over, so that the controller plans against a disturbance it expects to meet rather than cancelling the one present now. Chee et al. [3] embed a learned component inside the prediction constraint of a nonlinear MPC; Li et al. [12] inject a predicted downwash-force sequence into the optimiser's prediction model; and Hsieh et al. [7] add an $\mathcal{L}_1$ online estimate to the same family. All three run the optimisation off-board or on a companion computer rather than on the vehicle. This injection point is not among the strategies evaluated here — its onboard feasibility on a Crazyflie-class platform is itself open — but it is retained in this survey because its evaluation conditions form part of the argument in Section 1.8.

Inside an optimal feedback controller for a docking task. A further instance is worth separating because its task differs in kind. Shankar et al. [13] embed a learned lumped downwash force online in an infinite-horizon LQR acting on a feedforward-linearised follower, beneath an inner attitude loop, and use it for vertical docking: a leader–follower manoeuvre in which the vehicles close to within one or two body-lengths, so that the disturbance is not incidental to the task but constitutive of it. The platforms are two custom quadrotors of roughly 0.27 m diagonal and 0.7 kg, not the Crazyflie class, and the free-flight dock is performed at a vertical offset of about 0.36 m — close to one body-length — with a curriculum descending to around 0.5 m. The model itself is the $SO(2)$-equivariant one of their prior work, trained offline from about five minutes of data and queried online on an onboard Raspberry Pi, with pose from motion capture; the evaluation is hardware only. Reported accuracy depends on the manoeuvre: the abstract gives terminal error below 0.06 m, the body reports terminal vertical error typically below 0.1 m for a hovering leader, and for a moving leader below 0.05 m with the model against more than 0.2 m without it. Physical docks succeeded in eight of ten attempts against a hovering leader at 0.36 m; against a leader moving at 0.25 m/s success was limited, which the authors attribute to gripper latency. It is the closest published setting to the tightest scenarios of Chapter **??**, and the regime in which — on our reading — compensation ceases to be an accuracy improvement and becomes a precondition for the manoeuvre.

1.6 Learning the compensation directly

A third stance declines to model the force at all and learns the corrective action instead. Zhang et al. [20] train a residual reinforcement-learning module on top of a cascaded con-

troller, compensating the disturbance and thrust loss caused by downwash without forming an explicit force estimate at any point. Two properties distinguish the stance sharply from Section 1.4. The policy observes only tracking error, the baseline controller's command and its own previous action — no neighbour state and no inter-vehicle communication — which removes a bandwidth requirement and a failure mode, at the cost of being unable to anticipate a neighbour it cannot perceive. And giving up the explicit force estimate removes the predicted-against-measured comparison that every other learned method here is evaluated by, which makes failures correspondingly harder to attribute.

This family is not among the strategies evaluated here, for that second reason as much as for scope: the evaluation built in Chapter **??** rests on the residual being a measurable quantity, and a method that never forms one is assessed on tracking performance alone. It remains the most natural direction in which to extend the comparison.

1.7 Hybrid reactive and predictive compensation

Measurement and prediction are complementary in principle. Prediction is available before the disturbance acts but is valid only over the configurations its training data covered; measurement is general but delayed. Combining them is therefore natural, and has been done for a single vehicle.

Cobo-Briesewitz et al. [4] propose two methods. In the first, LINDI, a neural network replaces INDI's residual estimate entirely, trained on smoothed data obtained by spline fitting. In the second, NA-INDI, the total residual is written $f_a = f_{a,\mathrm{NN}} + f_{a,\mathrm{INDI}}$: the network predicts a component and the incremental term reasons about the remaining mismatch. Both are evaluated on a single quadrotor, with and without a slung payload, on figure-of-eight and circular trajectories, neither of which was in the training set.

The headline result concerns sensing rather than the hybrid: a network can generate smooth approximations of INDI's output without requiring rotor-speed sensors at LINDI's runtime, which removes the hardware prerequisite identified in Section 1.3 from that method's deployment — though rotor speeds are still needed to produce the training labels, and NA-INDI retains an incremental branch that uses them. Without a payload, residual compensation of any kind reduces tracking error by around 40 % relative to the geometric baseline; with a payload the reductions are smaller, in the range of 19–24 %. The hybrid's own advantage is real but modest: without a payload it gives the best performance, though the authors describe the margin as small, and with a payload the authors describe it as remaining *similar* to INDI alone. They also observe a slight drop in performance on the circular trajectory — for LINDI, in the no-payload case — attributing it to that trajectory being out of distribution relative to the training data, a direct instance of the limitation that defines the predictive family.

Two features of this setting bear on whether the result transfers. The residual there is self-induced, blade flapping, drag, the swing of a payload, and is predicted from the vehicle's own state. In a multi-robot team the residual is produced by other vehicles and is a function of a measurable relative state, which gives the predictive component an input the single-vehicle setting does not provide. Whether that turns a small margin into a decisive one is untested, and is one of the questions this thesis addresses.

1.8 How these methods are evaluated

The preceding sections describe a field with strong results. Reported improvements include a factor of two or better, up to four, in worst-case height error [15], up to a threefold

Table 1.1: Evaluation conditions across the interaction-force literature. Each result is reported against that work's own baseline, and no two rows share a platform, a team size, a separation and a trajectory class.

Work	Team size	Separation	Reported
Tal and Karaman [18]	1	n/a (no formation)	aggressive
Smeur et al. [16]	1	n/a	jet / GNS
Shi et al. [15]	2–4 train, 2–5 test	0.20–0.25 m vert.	max. $\|z\|$ e
Neural-Swarm2 [14]	16, heterogeneous	24 cm vert.	worst-case
Smith et al. [17]	2	0.6–0.8 m eval. (train. to ≈0.5 m)	3D and ve
Gielis et al. [5]	up to 4 total (rig sufferer + ≤ 3 flyers)	test rig, sufferer not in free flight	force-pred
Yang et al. [19]	1, planar (sim. only)	n/a	simulation
Hsieh et al. [8]	2	0.3 m and 0.4 m vert.‡	RMSE an
Chee et al. [3]	2, Crazyflie 2.1	commanded 0.2–0.4 m; tight case ≈0.08 m	RMSE / r nominal M
Li et al. [12]	2, custom	neighbour-relative	NMPC wi force sequ
Hsieh et al. [7]	3	V-stack and I-stack; pairwise 0.2 m	tracking v
Shankar et al. [13]	2, custom (≈0.27 m)	dock ≈0.36 m	terminal v dock succ
Cobo-Briesewitz et al. [4]	1 (+ payload)	n/a	position t
Zhang et al. [20]	2, heterogeneous	0.50 m vert. hover/circle; dock to 0.10 m, release at 0.05 m	position a

† The sixteen-robot flight and the threefold reduction are separate reports; on the sixteen-robot flight the body gives a factor of about 1.5. ‡ Hardware tracking comparisons are at commanded $\Delta z = 0.3\,m$ and 0.4 m; the 0.2–0.5 m range belongs to the simulation and prediction evaluation.

reduction in worst-case tracking error against a zero-interaction baseline [14], 36 % and 56 % improvements in three-dimensional and vertical tracking [17], 31 % against nominal baselines as stated in the abstract [8], and roughly 40 % for residual compensation without a payload [4]. Taken individually, each is credible and internally validated.

They cannot, however, be composed. Table 1.1 lists the conditions under which they were obtained.

A 36 % improvement measured on two vehicles in close proximity and a 31 % improvement measured in a tight formation are not the same quantity: they differ in what was flown, on what airframe, at what separation, against what baseline, and under what metric. Neither can be subtracted from the other, and their ordering carries no information about which method would prevail if both were flown under the same conditions. The difficulty compounds where the baselines themselves differ, since an improvement relative to a weaker baseline is easier to obtain.

This is not a criticism of any individual work. Each is internally valid, and there is no convention in the field that any of them violated. It is a limitation of the literature taken as a whole: the results are not designed to compose, and no amount of careful reading assembles them into a ranking. Establishing one requires an experiment built for the purpose, which is what distinguishes a comparison from a survey.

Comparisons that do exist The field does conduct systematic comparisons, and two recent ones bound what remains open.

Gu et al. [6] benchmark four online-learning uncertainty estimators — extreme learning machines, radial-basis-function networks, echo state networks and Gaussian processes —

inside a single modular framework in which a nominal controller is paired with a swappable residual-learning module. They treat the uncertainty as a lumped additive wrench (f_a, τ_a) recovered by inverting the discrete Newton–Euler dynamics — analogous to the residual acceleration used here, though not identical to it a priori. They evaluate estimation accuracy, tracking performance and computational cost, and reach the kind of conclusion a comparison should: not a winner, but a set of trade-offs, with Gaussian processes most accurate yet limited by cubic training complexity, the lightweight parametric models fast but sensitive to initialisation, and echo state networks the most balanced.

Two things separate it from the present work. Its subject is a *single* quadrotor under generic uncertainty, and all four methods learn *online*; it therefore compares estimators *within* one family rather than the families of Sections 1.3 to 1.7. And its evaluation is in simulation throughout: the reported hardware component is the learning modules executing on an embedded computer in the loop with a simulator, which measures latency rather than flight, and the authors list validation on a real quadrotor as future work.

Its approach to fairness is nonetheless directly relevant. They tune the parametric models — the extreme learning machine, the radial-basis-function network and the echo state network — to their own optima by Bayesian optimisation under otherwise consistent conditions, so that each operates near its best achievable performance. This thesis tunes each compared strategy on the hardware gate and then freezes those gains before the comparison and training-collection flights; gains are not required to be identical across families. Both answer the same objection — that an observed difference might reflect tuning effort rather than method — and Section **??** returns to why the choice is genuinely open.

Abro et al. [1] compare fractional-order PID, fractional-order LQR and integral state feedback on close-proximity flight, explicitly motivated by downwash and by the excessive safety distances that neglecting it imposes.[2] This is a controller comparison in our setting: multiple vehicles, and downwash-aware. Of the three, only integral state feedback is classical; the fractional-order laws are not. All three are nonetheless feedback regulators: they do not present a learned or first-principles predictor of the interaction force in the sense of the works above, so the reactive, predictive and hybrid families remain uncompared against one another.

Together these delimit the gap precisely. It is not that close-proximity control is uncompared, nor that the field lacks the methodology. It is that the specific families defined by *how they obtain information about the interaction force*, measure it, predict it, or both, have not been placed under common conditions.

Such an experiment has requirements that follow directly from the failure modes above: a single airframe; identical trajectories and formations across all methods; per-strategy gains frozen after tuning before data collection, so that a difference cannot be a tuning artefact; a single definition of the residual, measured identically under every controller including those that do not use it; and a null condition in which no interaction is present, to establish the measurement floor and so distinguish a real effect from noise. Chapter **??** describes the resulting design.

1.9 Summary and gap

A controller can obtain information about the interaction force in two ways. It can measure the residual online, which requires no prior model and generalises to disturbances never

[2] Cited for the methods compared and for that motivation; the abstract's quantitative improvement figure and its claim about larger swarms are not used here.

seen, but is delayed and, in its established form, depends on rotor-speed sensing. Or it can predict the residual from the relative states of its neighbours using a model learned offline, which acts ahead of the disturbance but is valid only where the training data reached. A third approach learns the corrective action directly, forgoing an explicit force estimate and, in at least one instance, inter-vehicle communication as well. Representative methods exist for each, and each is individually validated.

Three gaps follow from this survey.

First, and principally, to the best of our knowledge the reactive, predictive and hybrid families have not been evaluated against one another under identical conditions. Systematic comparisons in adjacent territory exist — of online-learning estimators on a single vehicle, and of classical regulators for close-proximity swarms (Section 1.8) — but none spans the families distinguished by how they obtain the interaction force. The individual results are strong; what the field lacks is not strong results but comparable ones, and its reporting conventions — improvements against each work's own baseline, on its own platform, at its own separations — make the comparison impossible to assemble after the fact. This thesis compares the uncompensated baseline, reactive compensation (INDI) and predictive compensation (geometric+NN, and FBL+NN if implemented). Multi-robot evaluation of a reactive–predictive hybrid is outside the scope of this thesis, as already stated above.

Second, the hybrid of measurement and prediction has been demonstrated only for a single vehicle, where the residual is self-induced and its advantage was described as small without a payload and as leaving performance similar to the reactive method with one [4]. The multi-robot case, in which the residual is larger and is a function of a measurable relative state, remains untested for such hybrids; evaluating them on teams is outside the scope of this thesis.

Third, the summation used to aggregate per-neighbour predictions is known to be an approximation [15, 5], yet the magnitude of the penalty it carries for a deployed model, evaluated on teams larger than those it was trained on, is not reported.

Provenance, not pending work. Supported by two audits: an arXiv keyword sweep, and a cross-database sweep over OpenAlex, Crossref and DBLP including a citation-graph scan of 157 works citing the Neural-Swarm line. Both found adjacent comparisons but none spanning these families. Neither is exhaustive; keep the "to the best of our knowledge" phrasing.

Chapter **??** formalises the residual wrench these methods estimate or predict, and Chapter **??** describes the strategies compared in this thesis and the point at which each injects its knowledge of the interaction into the control law.

Bibliography

[1] Ghulam E Mustafa Abro, J. Alsayegh, and Hifza Mustafa. High-precision uav swarm control: Evaluating fractional-order methods for close-proximity operations. *IEEE Journal of Emerging and Selected Topics in Industrial Electronics*, 2025. Abstract read via OpenAlex; full text not held (paywalled).

[2] Leonard Bauersfeld, Koen Muller, Dominic Ziegler, Filippo Coletti, and Davide Scaramuzza. Robotics meets fluid dynamics: A characterization of the induced airflow below a quadrotor as a turbulent jet. arXiv:2403.13321, 2024. Published in IEEE Robotics and Automation Letters.

[3] Kong Yao Chee, Pei-An Hsieh, George J. Pappas, and M. Ani Hsieh. Flying quadrotors in tight formations using learning-based model predictive control. arXiv:2410.09727, 2024.

[4] Eckart Cobo-Briesewitz, Khaled Wahba, and Wolfgang Hönig. Learned incremental nonlinear dynamic inversion for quadrotors with and without slung payloads. arXiv:2503.09441, 2025. Published in Proceedings of the 8th Annual Conference on Learning for Dynamics and Control (L4DC), PMLR.

[5] Jennifer Gielis, Ajay Shankar, Ryan Kortvelesy, and Amanda Prorok. Modeling aggregate downwash forces for dense multirotor flight. arXiv:2312.03488, 2023.

[6] Weibin Gu, Jiance Zhao, and Alessandro Rizzo. Learning uncertainties online for quadrotor flight control: A comparative study. *Journal of Intelligent & Robotic Systems*, 2025. Abstract read via OpenAlex; full text not held (paywalled).

[7] Pei-An Hsieh, Kong Yao Chee, and M. Ani Hsieh. Online adaptation for flying quadrotors in tight formations. arXiv:2506.17488, 2025.

[8] Pei-An Hsieh, Fengjun Yang, Nikolai Matni, and M. Ani Hsieh. Flatness-preserving residual learning for real-time tight quadrotor formation flight. arXiv:2607.12275, 2026.

[9] Anoop Kiran, Nora Ayanian, and Kenneth Breuer. Influence of static and dynamic downwash interactions on multi-quadrotor systems. arXiv:2507.09463, 2025.

[10] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Control of complex maneuvers for a quadrotor uav using geometric methods on se(3). arXiv:1003.2005, 2010. COMPANION arXiv paper. Used for equation-level verification of the objects shared with CDC 2010: the attitude error function, the attitude and angular velocity errors, and the full torque law with feedforward. Cite CDC 2010 as primary for Strategy 0. The extended v4 revision, and only it, carries the three flight modes, velocity mode, hybrid concatenation, the heading-projection proposition and the switched

720/360-degree example; cite THIS key, not CDC, for any of those. Do not copy v4 region-of-attraction bounds or Lyapunov matrices onto the CDC citation.

[11] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010. PRIMARY reference for the geometric SE(3) controller (Strategy 0). Tracks position and one heading direction, four outputs. Its torque law is CDC eq. (16) and includes the angular feedforward term; the firmware implements a reduced form of it. Stability is regional (Prop. 2 exponential below 90 degrees; Prop. 3 almost-global attractiveness below 180 degrees), not global. Its companion is arXiv:1003.2005v1. Do NOT attribute the three flight modes, velocity mode, heading-projection proposition or the switched 720/360-degree example to this paper; those are in the v4 companion only.

[12] Jinjie Li, Liang Han, Haoyang Yu, Yuheng Lin, Qingdong Li, and Zhang Ren. Nonlinear mpc for quadrotors in close-proximity flight with neural network downwash prediction. arXiv:2304.07794, 2023.

[13] Ajay Shankar, Heedo Woo, and Amanda Prorok. Docking multirotors in close proximity using learnt downwash models. arXiv:2311.13988, 2023.

[14] Guanya Shi, Wolfgang Hönig, Xichen Shi, Yisong Yue, and Soon-Jo Chung. Neural-swarm2: Planning and control of heterogeneous multirotor swarms using learned interactions. arXiv:2012.05457, 2020. Published in IEEE Transactions on Robotics.

[15] Guanya Shi, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung. Neural-swarm: Decentralized close-proximity multirotor control using learned interactions. arXiv:2003.02992, 2020.

[16] Ewoud J. J. Smeur, Guido C. H. E. de Croon, and Qiping Chu. Cascaded incremental nonlinear dynamic inversion control for mav disturbance rejection. arXiv:1701.07254, 2017.

[17] H. Smith, A. Shankar, J. Gielis, J. Blumenkamp, and A. Prorok. So(2)-equivariant downwash models for close proximity flight. arXiv:2305.18983, 2023.

[18] Ezra Tal and Sertac Karaman. Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness. arXiv:1809.04048, 2018.

[19] Fengjun Yang, Jake Welde, and Nikolai Matni. Learning flatness-preserving residuals for pure-feedback systems. arXiv:2504.04324, 2025.

[20] Ruiqi Zhang, Dingqi Zhang, and Mark W. Mueller. Proxfly: Robust control for close proximity quadcopter flight via residual reinforcement learning. arXiv:2409.13193, 2024.